

Your SBOM stops at the **model**.

Supply chain security when the dependency reasons.

Emmy Eide · Engineering & Cybersecurity Lead, Realign Labs

Conf42 DevSecOps 2026

THE NUMBER

6 of 7

vulnerabilities in a project come from dependencies the team didn't write.

Keynote, CloudNativeSecurityCon North America 2023 (Red Hat)

Silence isn't the absence of risk. It's the absence of a decision.

Every risk your pipeline accepts, should be actively accepted by somebody. This risk gets accepted by no one, because “green” doesn't generate the opportunity.

WHAT WE GOT RIGHT

We built the machine.

Scan

Sign

SBOM

Gate

Enumerate. Sign. Attest. Gate.

Every one of them assumes one thing: **the artifact is code you can read.**

THE ARTIFACT CHANGED

Two ways a model gets into your product.

You host the weights

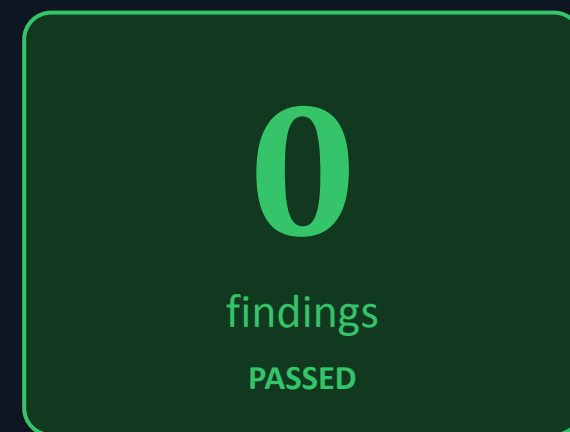
A multi-gigabyte blob in your image and nothing in your pipeline reads it for behavior.

You call an API

No blob at all. The model runs on someone else's infrastructure where a version can change under you with no artifact change, no SBOM diff, nothing.

It left your artifact. **The responsibility didn't.**

**No findings
renders identically to clean.**



A scanner that can't read an artifact returns no findings. There's no amber, no 'unknown' — and zero is the same zero a genuinely clean artifact returns.

Read what it actually caught.

Seven clusters

- Metadata
- System-level properties
- Models
- Datasets
- Infrastructure
- Security properties
- Key performance indicators

The one thing they left out

autonomy : what the thing can do on its own.

“it was decided to not explicitly call it out as a separate element.”

Software Bill of Materials for AI — Minimum Elements · G7, 2026 · p.23

An SBOM catalogs what a thing IS. This is the first artifact where what it is no longer tells you what it will do.

Which gates fire, which go quiet.

Gate	On a model-bearing artifact	Failure mode
Dependency / CVE scan	Parses the wrapper, not the weights	Silence — reads as clean
SBOM generation	Describes the packaging	Incompleteness as completeness
License compliance	Sees serving code, not the model's terms	Wrong scope
Signing / provenance	Still fires, still necessary	Attests who built it, not what it does
Admission control	Passes on a signed artifact	Right answer, wrong question
Runtime	Where the whole burden lands	Unbudgeted

Three questions. Two have products.



Is this file weaponized?

Solved enough to be a purchasing decision.



Has it been tampered with?

Partial : real work, needs the model file, an arms race.



What will it do when it works as intended?

Not a scanning problem. No static answer.

Three things you can do this quarter.

1 **Ask for an SBOM for AI against the G7 minimum elements.**

Then ask the questions it doesn't answer: which fields are attested vs. declared? what's the autonomy level and who owns it? what does your pipeline DO with any of it?

2 **Keep signing and be precise about what it buys.**

It proves origin, never behavior. If you invest in one new thing, invest in evidence generated while the thing is running.

3 **Be honest about the open problems.**

We can't verify most of what an AI SBOM asserts, there's no agreed unit for autonomy, and a passing eval isn't a passing regression test.

THE ASK

If you forget everything else, do two things.

1 Point your scanner at a model-bearing image.

Not whether it says pass. Whether it read the thing at all.

2 Write the risk acceptance. A name, and a date.

You can't write it without answering what the thing is allowed to do on its own. Writing it down creates the person who owns that.

Three things to leave with.

1. **The absence of a control produces silence, not an error.**
2. **The presence of a field is not the presence of a control.**
3. **Silence isn't the absence of risk — it's the absence of a decision.**